arXiv:2402.11078v3 [cs.LG] 3 Jun 2024

# Model Editing by Standard Fine-Tuning

**Govind Gangadhar** and **Karl Stratos**
Department of Computer Science
Rutgers University
{govind.gangadhar, karl.stratos}@rutgers.edu

## Abstract

Standard fine-tuning is considered not as effective as specialized methods for model editing due to its comparatively poor performance. However, it is simple, agnostic to the architectural details of the model being edited, and able to leverage advances in standard training techniques with no additional work (e.g., black-box PEFT for computational efficiency), making it an appealing choice for a model editor. In this work, we show that standard fine-tuning alone can yield competitive model editing performance with two minor modifications. First, we optimize the conditional likelihood rather than the full likelihood. Second, in addition to the typical practice of training on randomly paraphrased edit prompts to encourage generalization, we also train on random or similar unedited facts to encourage locality. Our experiments on the ZsRE and COUNTERFACT datasets demonstrate that these simple modifications allow standard fine-tuning to match or outperform highly specialized editors in terms of edit score.

## 1 Introduction

Model editing is a promising approach to combating incorrect or otherwise unwanted knowledge in LLMs. Given a set of edits that assert desired information, the approach aims to alter the model so that it not only succeeds in memorizing these edits (efficacy) but also applies the asserted information to new prompts (generalization), *without* changing inferences that should remain unchanged (locality). There is clearly a trade-off between these metrics. At one extreme, the model can achieve high efficacy by memorizing the edits, but it will fail in generalization and locality. At another extreme, the model can achieve high locality by staying unmodified, but it will fail in efficacy and generalization.

Naively fine-tuning the model on the requested edits is well known to perform poorly, especially in locality. This motivated researchers to develop highly specialized model editors (Mitchell et al., 2022; Meng et al., 2022, 2023; Hartvigsen et al., 2023; Li et al., 2024a, *inter alia*). A central theme of these methods is "minimally invasive editing" achieved by a careful selection of layers, model-specific adapter designs, and low-rank updates. While technically interesting, they require a suite of assumptions which may not be satisfied in other contexts (e.g., with different model architectures). In contrast, standard fine-tuning is simple, completely agnostic to the details of the model, and can take advantage of advances in standard training such as parameter-efficient fine-tuning (PEFT) techniques for immediate computational efficiency.[1]

| Editor | Standard model? | Batched edits? | No extra training? | Effective edits? |
|---|---|---|---|---|
| Naive fine-tuning | ✓ | ✓ | ✓ | ✗ |
| MEND | ✗ | ✓ | ✗ | ✓ |
| ROME | ✗ | ✗ | ✓ | ✓ |
| MEMIT | ✗ | ✓ | ✓ | ✓ |
| Our fine-tuning | ✓ | ✓ | ✗ | ✓ |

Table 1: Conceptual comparisons of model editors.

In this work, we show that standard fine-tuning can yield competitive model editing performance, focusing primarily on improving the edit score. We shift the focus from models and algorithms to training objectives and data augmentation. See Table 1 for conceptual comparisons between our approach and other editors. Our fine-tuning uses two small but impactful modifications. First, in line with the theme of minimal editing, we optimize the *conditional* likelihood (i.e., mask all tokens except the edited target). Second, we augment the training data with random or similar unedited facts to

[1]We emphasize that our use of PEFT is only for computational convenience (i.e., we use black-box PEFT and are not concerned with its details such as LoRA vs others) and should not be confused with works that develop specialized adapters for model editing such as Yu et al. (2024).

encourage locality. This is in addition to the typical practice in existing works of training on randomly paraphrased edits to encourage generalization. These simple modifications allow standard fine-tuning to match or outperform specialized editors in mass-editing, and also perform respectably in single-editing.

## 2 Task

Our main task is mass-editing (i.e., performing multiple edits at the same time). Let $\mathcal{V}$ denote the vocabulary. A **fact** is a sentence $x \in \mathcal{V}^T$ that expresses a subject-relation-object triple $(s, r, o)$ in natural language. We follow the convention in the model editing literature and assume that the object $o \in \mathcal{V}^m$ is the last $m$ tokens of $x$. The prefix $\pi = (x_1 \dots x_{m-1})$ expresses $(s, r)$ and is denoted as the **prompt**. Let $\mathcal{E}$ denote a set of facts to enforce (i.e., requested edits). Our goal is to edit a language model so that it upholds the relations expressed in $\mathcal{E}$ without changing its behavior on other facts. In ZsRE, $\mathcal{E}$ consists of 10,000 factual statements (e.g., "The artwork Gideon's Way was by who? John Creasey"). In COUNTERFACT, $\mathcal{E}$ consists of 10,000 counterfactual statements (e.g., "TextEdit, a product of Nintendo").

An editor is evaluated by three competing metrics: efficacy, generalization, and locality. Let $p_\theta$ denote a language model edited on $\mathcal{E}$. In ZsRE, efficacy is the accuracy of $o = \arg\max_y p_\theta(y|\pi)$ for $(\pi, o) \in \mathcal{E}$; generalization is the accuracy of $o = \arg\max_y p_\theta(y|\pi_{\text{par}})$ where $\pi_{\text{par}}$ is a paraphrase of $\pi$; locality is the accuracy of $o_{\text{unrel}} = \arg\max_y p_\theta(y|\pi_{\text{unrel}})$ where $(\pi_{\text{unrel}}, o_{\text{unrel}})$ is an unrelated fact. In COUNTERFACT, efficacy and generalization measure the accuracy of $p_\theta(o|\pi) > p_\theta(o_{\text{pre}}|\pi)$ and $p_\theta(o|\pi_{\text{par}}) > p_\theta(o_{\text{pre}}|\pi_{\text{par}})$ where $o_{\text{pre}}$ is a pre-edit object for $(\pi, o) \in \mathcal{E}$ (e.g., "Apple" in "TextEdit, a product of Nintendo"). Locality measures the accuracy of $p_\theta(o_{\text{pre}}|\pi_{\text{nb}}) > p_\theta(o|\pi_{\text{nb}})$ where $\pi_{\text{nb}}$ is a "neighborhood" unedited prompt whose target is $o_{\text{pre}}$ (e.g., "Macintosh File System, a product of" $\rightarrow$ "Apple"). In both datasets, the final **edit score** is the harmonic mean of efficacy, generalization, and locality.

It is well known that naive fine-tuning, namely just optimizing

$$\min_\theta -\sum_{x \in \mathcal{E}} \log p_\theta(x) \tag{1}$$

results in a poor edit score. The main reason is that while it improves efficacy and possibly even generalization, it harms locality by changing the model's predictions on unrelated or neighborhood prompts. This led to the development of highly specialized editing methods, with the implicit assumption that fine-tuning is not effective for model editing.

Fact: The mother tongue of Danielle Darrieux is English

Paraphrase augmentation (generated by the model):
1) *The present invention relates.* The mother tongue of Danielle Darrieux is English

Random fact augmentation (from the training split):
1) Crate & Barrel was founded in Chicago
2) Haines Borough is within Nevada

Figure 1: An example from COUNTERFACT along with paraphrase and random fact augmentation.

## 3 Method

We propose a purely fine-tuning-based method that achieves a competitive edit score. Our objective is a slight variation of (1):

$$\min_\theta -\sum_{(\pi,o) \in \mathcal{E} \cup \mathcal{P} \cup \mathcal{R}} \log p_\theta(o|\pi) \tag{2}$$

There are two small but important differences between (1) and (2). First, we optimize the *conditional* likelihood of the edit target $o|\pi$ rather than the full likelihood. Our motivation is to make the training more focused (for efficacy and generalization) and minimize the damage caused by fine-tuning (for locality).

Second, we fine-tune not only on the requested edits $\mathcal{E}$ but also additional facts in $\mathcal{P}$ and $\mathcal{R}$ to promote generalization and locality. $\mathcal{P}$ is pseudo-paraphrases of the prompts in $\mathcal{E}$ obtained by prepending random words generated from the (unedited) model. Paraphrase augmentation is universally used in existing model editors (Mitchell et al., 2022; Meng et al., 2023; Li et al., 2024a). Additionally, we augment with facts $\mathcal{R}$ that should not be altered by editing on $\mathcal{E}$. There are many ways to obtain $\mathcal{R}$, but we find taking *random* facts from the training split to be simple and effective. We filter $x \in \mathcal{R}$ to ensure that $x$ does not include the subject-relation-object triple used in evaluation. Figure 1 shows an example from COUNTERFACT (we only show 1 paraphrase and 2 random facts for illustration, in practice we use more).

We emphasize that the augmented sets $\mathcal{P}$ and $\mathcal{R}$ do not use the evaluation facts (otherwise it is trivial). Given the requested edits $\mathcal{E}$, we perform the data augmentation and fine-tune the model according to (2).

**Contrastive learning.** Given that our goal in COUNTERFACT is to achieve $p_\theta(o|\pi) > p_\theta(o_{\text{pre}}|\pi)$ where $o$ and $o_{\text{pre}}$ are the new and pre-edit objects for a given prompt $\pi$, it is natural to consider a contrastive objective. We experimented with DPO (Rafailov et al., 2023) where we frame $o|\pi$ as the "preferred" response over $o_{\text{pre}}|\pi$, optimizing the DPO loss jointly with (2). However, we found that prompt masking and data augmentation are already effective and do not benefit from contrastive learning.

## 4 Related Work

We discuss existing model editors and the different assumptions they require to highlight the simplicity of our approach; we refer to Zhang et al. (2024) for an in-depth survey.

MEND (Mitchell et al., 2022) is a meta-learning method (Sinitsin et al., 2020; De Cao et al., 2021) that predicts the change in the gradient. Similar to our approach, it also involves an explicit training stage (on the training split of ZsRE). It uses a special rank-one decomposition of the gradient for parameter efficiency. In our case, we achieve parameter efficiency without special considerations simply by leveraging black-box PEFT.

ROME (Meng et al., 2022) is a locate-then-edit method (Geva et al., 2021; Dai et al., 2022) that first uses causal tracing to identify the feedforward layer to change, and then applies a rank-one update to the layer's weight matrix. While it does not involve an explicit training stage, it does require an explicit knowledge of the subject $s$ in the edit and its vector representation (which is heuristically induced). It also relies on Wikipedia to estimate the covariance matrix of the subject embeddings (needed for the rank-one update). MEMIT (Meng et al., 2023) extends ROME to multiple edits by carefully spreading updates to multiple layers. Like ROME, it requires layer specification, subject embeddings, and covariance statistics obtained from Wikipedia.

IKE (Zheng et al., 2023) proposes in-context learning for model editing. While similar to our approach in avoiding the need to develop a specialized editor, it requires a strong model capable of effective in-context learning and is critically limited to single-editing. Consequently, IKE focuses on single-editing experiments with large-scale LLMs, making their results not comparable to ours.

We define our scope as achieving a competitive edit score through standard fine-tuning. In particular, we do not focus on preserving general capabilities of the model being edited. Recent work shows that performance on a variety of downstream tasks drops to zero under any editor (Gu et al., 2024). We leave preserving the general capabilities of an LLM with a fine-tuning editor as the next step of our work.

## 5 Experiments

### 5.1 Mass-Editing

Our main results are on mass-editing with on ZsRE and COUNTERFACT, following the same setting in Meng et al. (2023). Each dataset provides 10,000 requested edits (i.e., $\mathcal{E}$). As described in Section 3, we augment the fine-tuning data with pseudo-paraphrases of the edit prompts $\mathcal{P}$ for generalization supervision and random facts $\mathcal{R}$ for locality supervision. More specifically, for each edit in $\mathcal{E}$ we generate 15 paraphrases and take 20 facts from the training split while ensuring that they do not contain any evaluation facts. Thus the total number of facts we fine-tune on is 360,000. We fine-tune GPT-J (6B) (Wang and Komatsuzaki, 2021) with LoRA (Hu et al., 2022) for computational efficiency. We optimize (2). The training takes around 2–2.5 hours on 8 GPUs.[2]

We give the results in Table 2 where the row "FT + Mask + Para + Rand" corresponds to our final method. We can make the following observations. First, the proposed prompt masking in (2) (FT + Mask) improves the performance of vanilla fine-tuning (FT). In fact, on ZsRE this already outperforms MEMIT without any data augmentation. Second, augmenting the data with paraphrased prompts substantially improves generalization (FT + Mask + Para). Third, augmenting the data with random facts further improves locality (FT + Mask + Para + Rand), allowing standard fine-tuning to match the performance of MEMIT on COUNTERFACT. Adding the DPO loss on top of the final method does not yield improvements.

We perform additional experiments on the WikiRecent dataset (Cohen et al., 2023) with similar conclusions; details are given in Appendix A.

### 5.2 Single-Editing

To compare with existing methods developed for single-editing (i.e., updating the model for a single fact), we consider optimizing (2) one edit at a time.

[2]The code is available at: https://github.com/au-revoir/model-editing-ft.